

UNIVERSITÀ DI PISA

DIPARTIMENTO DI INFORMATICA

Corso di Laurea Triennale in Informatica

LLM-based Design Pattern Detection

Relatori:

Prof. Jacopo Soldani

Dr. Lucia Passaro

Candidato:

Alessandro Fiumanò

ANNO ACCADEMICO 2025/2026

Contents

1 Introduction **3**
1.1 Context and motivations 3
1.2 Objectives and contributions 4
1.3 Thesis Organization 5

2 Background **7**
2.1 Design Patterns 7
2.2 Large Language Models for Code Analysis 10
2.3 LLMs responses: Structured outputs 11
2.4 Ranking-based evaluation 11

3 Dataset **13**

4 Prototype **15**

5 Experiment **17**

6 Related work **19**

7 Conclusions **21**

Abstract

Thesis abstract

Chapter 1

Introduction

1.1 Context and motivations

Design pattern detection has become a crucial aspect of software engineering, as it supports quality assurance, technical debt management, code smell identification, and overall maintainability. This process is closely related to refactoring, a systematic method for improving code without changing its external behavior, enhancing its readability and structure.

Code smells typically arise from codebase growth or poor programming practices, acting as indicators of underlying issues that can be addressed during refactoring: while they are often easy to find and fix, they may be just symptoms of deeper architectural flaws. Technical debt refers to the maintenance cost growth due to suboptimal design or implementation choices: although in the short term they can speed up code development, over time they make the system more complex to understand, modify, test, and evolve.

Design patterns can be used as a means to deal with structural issues and guide refactoring decisions, as they represent a solution to common problems in software development. Since patterns are described at a high level of abstraction, there are several ways to choose and apply them, leading to different concrete implementations. While useful, design patterns should not be applied indiscriminately, since forcing their application in unsuitable contexts may lead to overengineering, as well as an increase in technical debt.

However, the identification and application of design patterns are not fully deterministic processes, since they rely on the project context, the considered abstraction level, and the developer interpretation. As a result, it may not be possible to establish a universally valid answer to the question of which design pattern to use: such an answer is inevitably influenced by subjective factors.

In particular, LLMs may help to solve this problem, since they can help automate and speed up refactoring, despite the subjective nature of the task. Design pattern detection requires, in fact, a semantic and contextual understanding of the source code: this need makes the use of such models particularly relevant, precisely because

they do not merely search for rigid patterns, but are able to analyze structure, class responsibilities, and the relationships between components. This process represents an application of artificial intelligence to a branch of software engineering, named AI4SE.

LLMs are promising tools, but their ability to support design pattern recommendation must be evaluated systematically in order to understand their reliability in realistic scenarios: a software engineer should be able to assess whether a model is suggesting an appropriate pattern. As mentioned before, the impossibility of establishing a fully universal answer makes the correctness of a recommended design pattern difficult to define; therefore, suitable performance indicators must be identified.

1.2 Objectives and contributions

The main goal of this thesis consists of exploring how effective LLM-based design pattern detection is today in realistic scenarios. In order to achieve this, the identification of a way to evaluate LLMs' performance is crucial because of the need to understand software architecture and suggest the best design pattern. This analysis will investigate local model configurations by identifying, for each chosen LLM, the best parameter settings and subjecting them to realistic scenarios. The applicability of a specific pattern depends on multiple factors, and it is not guaranteed: the models will be challenged by architectural ambiguity.

The main purpose in terms of implementation is to create an end-to-end tool, which will support refactoring by taking as input Python code and providing a refactored version based on a suggested design pattern. The work involves an extension of the LLM Design Pattern Refactoring project [1], which applies a previously selected design pattern to a given codebase by adding a component that suggests the most suitable design pattern for the input code.

This thesis makes the following contributions:

- Adding new code examples for design patterns that were not present in the dataset of the previous project. Each example has been provided with a refactored version, implementing a specific design pattern, and a baseline version, where the design pattern has been removed. Baseline and refactored versions are functionally equivalent.

- Labeling of each baseline example in the dataset with a list of applicable design patterns, ranked by applicability.

- Implementation of a tool for querying a local LLM, aimed at generating a list of design patterns ranked according to their applicability to the analyzed code. Each query is characterized by the specification of the LLM name, textual prompt, and model temperature.

- Evaluation of LLM predictions according to seven metrics designed to evaluate the ranking quality of model predictions.
- Conduct of an ablation study aimed at establishing which query yields the best results.

1.3 Thesis Organization

The thesis is structured as follows:

- *Chapter 2* provides the necessary concepts to understand design pattern definitions and how to interact with Large Language Models.
- *Chapter 3* describes the dataset, explaining the refactored examples and which design pattern they implement.
- *Chapter 4* introduces the LLM-based design pattern detection tool.
- *Chapter 5* defines the evaluation methods for model predictions and discusses the results in light of them.
- *Chapter 6* discusses related work.
- *Chapter 7* concludes the thesis by summarizing the main results, discussing the possibilities and limitations of the approach.

Chapter 2

Background

2.1 Design Patterns

Design patterns can be defined as a common design structure useful for creating reusable object-oriented designs. They are characterized by four essential elements [2]:

- The name provides a concise way to refer to the pattern and captures the intended application domain.
- The problem describes the considered design issues and the context.
- The solution is expressed as the elements that make up the design, their relationships, responsibilities, and collaborations.
- The consequences of applying the pattern and its and trade-offs.

This work starts from the original dataset of the LLM Design Pattern Refactoring project [1], which contained three examples for each of the following design patterns: Strategy, State, Decorator, Adapter, and Factory Method. Each example was provided in two versions: a refactored version implementing the corresponding design pattern and a baseline version, where the pattern had been removed.

Starting from this dataset, this project extends the set of considered design patterns by introducing five additional patterns, briefly described in the following paragraphs.

The first added pattern is Abstract Factory, which provides an interface for creating families of related or dependent objects without specifying their concrete classes [2].

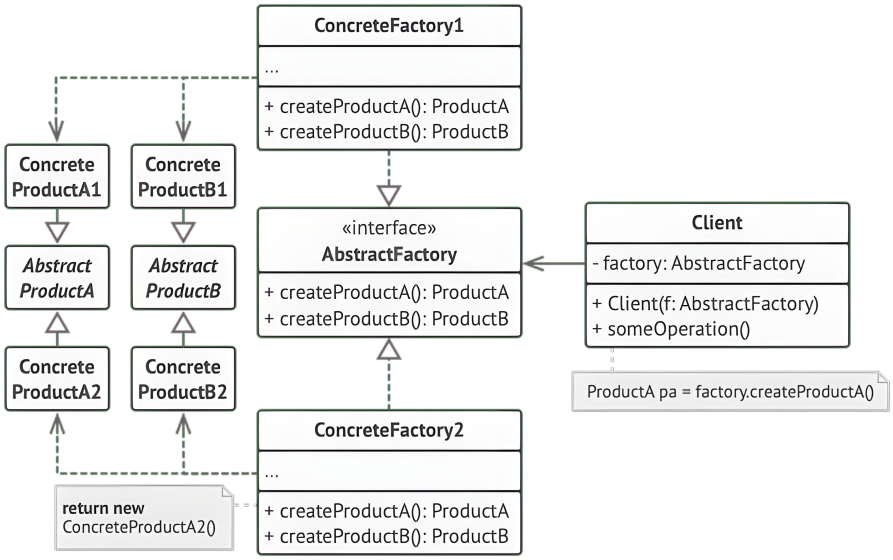


Figure 2.1: Abstract Factory pattern structure

Builder pattern separates the construction of a complex object from its representation so that the same construction process can create different representations [2].

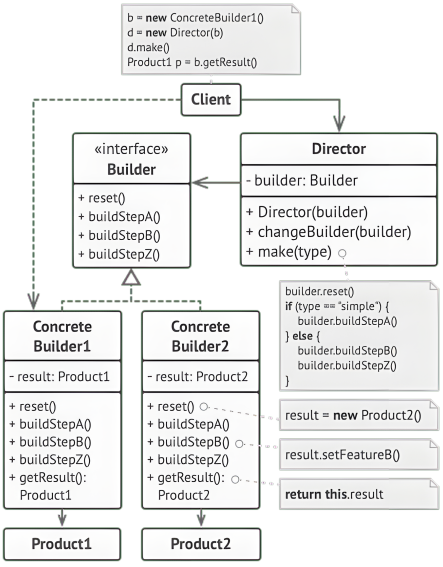


Figure 2.2: Builder pattern structure

Proxy pattern provides a surrogate or placeholder for another object to control access to it [2].

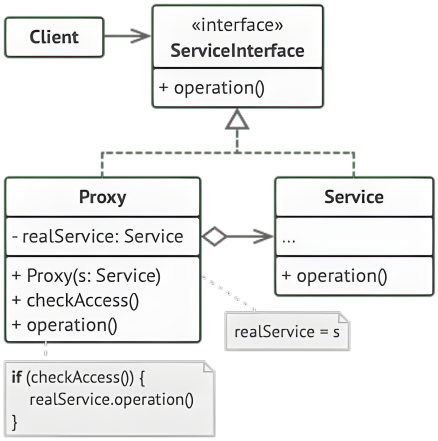


Figure 2.3: Proxy pattern structure

Command pattern encapsulates a request as an object, thereby letting you parameterize clients with different requests, queue or log requests, and support undoable operations [2].

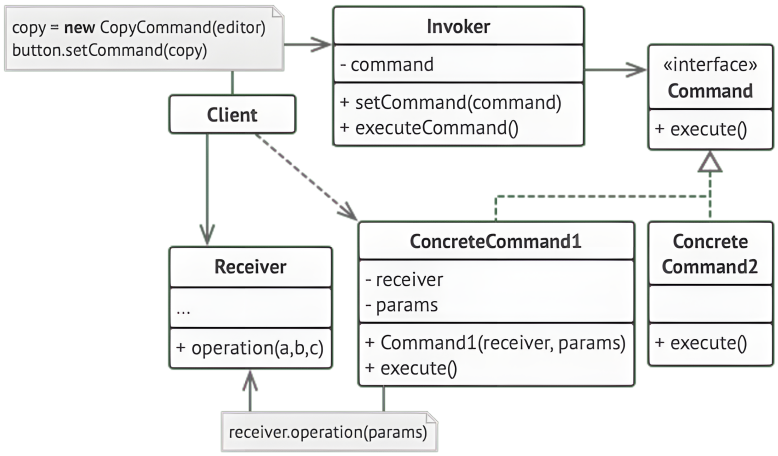


Figure 2.4: Command pattern structure

Template Method pattern defines the skeleton of an algorithm in an operation, deferring some steps to subclasses so that they can redefine certain steps without changing the algorithm's structure [2].

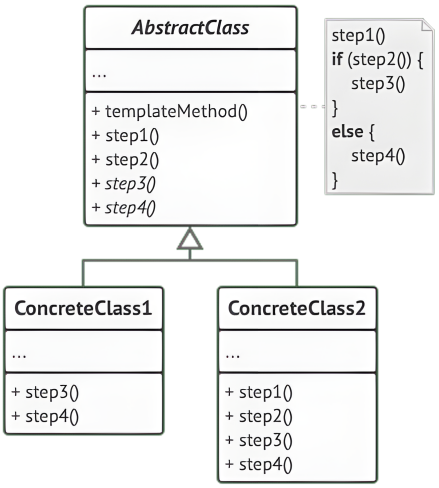


Figure 2.5: Template Method pattern structure

Each design pattern focuses on a particular object-oriented design problem or issue. It describes when it applies, whether it can be applied in view of other design constraints, and the consequences or trade-offs of its use. Design patterns are classified into three main categories, depending on their purpose:

- Creational patterns concern the process of object creation
- Structural patterns deal with the composition of classes or objects.
- Behavioral patterns characterize the ways in which classes or objects interact and distribute responsibility.

In this work, the considered design patterns cover all three categories. Strategy, State, Command, and Template Method belong to behavioral patterns; Decorator, Adapter, and Proxy belong to structural patterns; while Factory Method, Abstract Factory, and Builder belong to creational patterns.

2.2 Large Language Models for Code Analysis

What is a LLM

How to interact with a LLM: prompting and context learning (role + base + rules + output description + zero/few-shot)

Temperature and probabilistic generation

LLMs for code analysis: how they can be used to analyze code and suggest design patterns (risk analysis)

2.3 LLMs responses: Structured outputs

As mentioned in the previous section, LLMs generate responses based on a probabiistic distribution, except when their temperature is set to zero: as a result, when the same prompt is given multiple times, the generated response may differ across runs. This can be a problem when the expected output has a specific structure, such as an ordered list, and the model generates an output that does not conform to the expected structure. To address this issue, this work adopts a strategy to force the model to generate outputs that conform to the expected structure. This feature is named *Structured Outputs*, which ensures the model will always generate responses that adhere to a supplied description in JSON Schema [3]. [1]

2.4 Ranking-based evaluation

[1] JSON Schema is a declarative language for annotating and validating JSON documents' structure, constraints, and data types: it helps to standardize and define expectations for JSON data [4].

Chapter 3

Dataset

Chapter 4

Prototype

Chapter 5

Experiment

Chapter 6

Related work

Chapter 7

Conclusions

Bibliography

[1] Bartolomeo Zisa. *LLMDesignPatternRefactoring*. 2024. URL: https://github.com/Unipisa/LLMDesignPatternRefactoring/ (visited on 03/11/2026).

[2] Erich Gamma et al. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1995. ISBN: 978-0-201-63361-0.

[3] OpenAI. *Structured model outputs*. 2026. URL: https://developers.openai.com/api/docs/guides/structured-outputs (visited on 06/01/2026).

[4] JSON Schema. *JSON Schema Documentation*. 2026. URL: https://json-schema.org/docs (visited on 06/01/2026).